FineVideo VLA Pipeline

Technical Documentation for Colleagues

This document describes a data processing pipeline that transforms the **EmpathicRobotics/FineVideo-VLA-Agent** dataset into richly augmented training data for a Vision-Language-Action (VLA) model. The pipeline spans two Python scripts — **process_finevideo.py** and **decode_and_caption.py** — and produces pretraining data that teaches the model to observe, plan, act, and reflect as an embodied agent.

The core insight is that the robot is not a passive video watcher. It *sees* scenes (cosmos/seed2 visual tokens), *moves* through them (agent pose tokens), *plans* ahead using lookahead context, and *reflects* on what it has done. Every output record is framed from one of three perspectives — robot first-person, human first-person, or neutral cinematic — sampled randomly to maximise generalisation.

1. Source Dataset

The source is **EmpathicRobotics/FineVideo-VLA-Agent** on HuggingFace, a derived dataset built on top of FineVideo (HuggingFaceFV/finevideo). It contains 160 gzipped JSONL shards: 8 test and 152 train, totalling roughly 240 GB compressed.

Field	Description
global_context	Video-level description
scenes[]	List of scene objects
scene_title / scene_thematic_scene_metadata	Scene metadata
activities[]	List of activity objects per scene
video_tokens	Raw token string: cosmos, seed2, agent, avc tags
speech_transcript	ASR transcript for this activity window
text_prompt	Human-written scene description
props_present	Objects visible in the activity
video_editing	Camera/editing notes (jump cut, zoom, etc.)
time_range_sec	Start/end seconds of the activity clip
scene_id / activity_id	Unique identifiers
youtube_id	Derived from original_json_filename

Each top-level record maps to one YouTube video. Scenes and activities are nested inside it. The pipeline flattens this hierarchy so that each **activity** becomes one output record — with all parent context (video, scene) embedded as metadata and text.

2. Token Types

The video_tokens field contains a mix of several discrete token vocabularies, each wrapped in XML-style tags:

Token type	Meaning
<agent_N>	Body pose / motor tokens. The robot's own proprioceptive sequence. Represents what the robot did or will do.
<cosmos_N>	Cosmos-DV4x8x8 video tokens. Discrete indices into NVIDIA's causal video tokenizer. Decode to video frames
<seed_N>	Seed2 image tokens. Indices for the ontocord/seed2 image tokenizer. Decode to a single image via SD-unCLIP
<avc_N>	AVC tokens. Currently dropped (drop_avc=1.0).

Agent tokens are the most semantically important: they are the robot's own body. The pipeline frames them in three temporal tenses — **past** ('I performed…'), **future** ('I will perform…'), and **observe** ('I see a person performing…') — sampled at 45/35/20 weighting. When an agent token decoder is ready, the placeholder [motion sequence] in the code will be replaced with decoded natural-language motion descriptions.

3. Script 1 — process_finevideo.py

This script downloads each shard, decompresses it on the fly, runs four augmentation passes, and writes a flattened JSONL. The raw .gz is deleted immediately after processing to conserve disk space. A processed_files.txt tracker enables resumption after interruption.

3.1 Download & Decompress

Each file is fetched with wget from HuggingFace and decompressed via Python's gzip.open() in streaming mode — no intermediate uncompressed file is written to disk.

3.2 Four Augmentation Passes

Every shard is processed four times with different token-dropping configurations, producing 4× the output records from each source file:

Pass	drop_cosmos / drop_seed / drop_default
Pass 1	1.0 / 1.0 / 0.5 — agent tokens only
Pass 2	1.0 / 0.5 / 0.5 — agent + 50% seed
Pass 3	0.7 / 1.0 / 0.5 — 70% cosmos + agent
Pass 4	0.95 / 0.1 / 0.5 — mostly cosmos, 10% seed

3.3 Text Augmentation

Two augmentation techniques are applied to all human-readable text fields (transcripts, titles, keywords, context, structured fields):

- **WordNet synonym substitution** — words longer than 5 characters are looked up in the oewn:2020 WordNet via the wn SQLite package. A synonym is randomly selected and substituted, preserving capitalisation. Noun synsets are tried first, then verb synsets.
- **Stopword dropping** — common stopwords are dropped at a 5% rate, simulating natural variation in phrasing.
- **Chunk permutation** — sentences are split, chunked to ≤20 words, and 10% of chunks are randomly swapped in position.

3.4 Structured Field Rendering

Every rich metadata field is rendered as a labelled header block and added to the layout. Labels are selected from pools of synonymous phrases representing three framings:

Framing	Weight / Example label
Robot first-person	70% — 'objects_seen', 'individuals_observed', 'camera_motion', 'sequence_of_events'
Human first-person	20% — 'I see', 'I notice', 'I observe motion', 'What I see happening'

Cinematic (original)	10% — 'props', 'cast', 'video_editing', 'narrative_progression'

The label itself is then passed through augment_label(), which applies WordNet substitution at a 40% rate on individual words within the label. The separator between label and content is randomly chosen from ': ' ' - ' ' → '.

3.5 Lookahead & Planning Blocks

Each activity is processed with access to the **next activity** in the same scene (or the first activity of the next scene). This lookahead context enables forward-looking <think> blocks:

```
<think> Ok, I have seen Host stands at a desk holding the product.
My plan is to observe Host walking toward the camera. Let me plan it out by imagining the next scene.
</think>
```

Four temporal/planning block types are generated probabilistically:

Block type	Probability / Description
Plan (forward)	40% — lookahead think block: current → next
Reflect (backward)	35% — mood reflection on current scene
Mood-before	25% — scene preview line before token content
Instruction	25% — User: ... <think>...</think> framing

3.6 <think> Wrapping

50% of descriptive header blocks are wrapped in <think></think> tags with a randomly selected opener (23 options) and optional closer (12 options). Token content lines are never wrapped. This teaches the model to associate internal reasoning with observation.

```
<think> Analyzing the current scene. ### objects_seen: Camera harness. Canon cameras
I will act accordingly. </think>
```

3.7 Deduplication

A Python hash() of the final text string is stored in a per-file set. Duplicate outputs across passes are silently dropped before writing. The set is scoped per file to keep memory bounded.

3.8 Output Format

Each output line is a JSON object:

```
{"text": "...", "metadata": { ... }}
```

The metadata dict is a flat merge of all three levels — record, scene, activity — with the following fields always guaranteed:

- **youtube_id** — derived from original_json_filename
- **scene_id / scene_title / scene_mood / scene_thematic**
- **activity_id / time_range_sec**

- **speech_transcript** — original unaugmented transcript
- **text_prompt** — original scene description
- **props_present / video_editing**

4. Script 2 — decode_and_caption.py

This script reads the *_flattened.jsonl files produced by Script 1. For each record it extracts cosmos and seed2 token blocks, decodes them to visual content, captions the visuals with SmolVLM2, and patches the text with one of 14 framing modes. A parallel *_visuals.jsonl sub-dataset records every decoded image/video.

4.1 Model Loading

All three models load once at startup and are reused across all shards:

Model	Source / Notes
Cosmos-DV4x8x8 decoder	nvidia/Cosmos-0.1-Tokenizer-DV4x8x8 (JIT checkpoint). Installed via pip install cosmos-tokenizer.
Seed2 tokenizer	ontocord/seed2 (git clone). Custom Seed2Tokenizer class in seed2_tokenizer.py.
SD-unCLIP pipeline	stabilityai/stable-diffusion-2-1-unclip. Used by Seed2 as the image decoder UNet.
SmolVLM2-500M	HuggingFaceTB/SmolVLM2-500M-Video-Instruct. Captions both images and video frame sequences.

4.2 Decoding

Cosmos: indices are extracted from the token block with a regex, reshaped into the smallest valid (1, T, H, W) tensor where H and W are multiples of 8, decoded to a (3, T, H, W) float tensor, and converted to PIL frames. Up to 4 evenly-spaced frames are passed to the captioner.

Seed2: indices are extracted, formed into a (1, N) long tensor, and passed to Seed2Tokenizer.decode(pipe, tokens) which uses SD-unCLIP to generate a deterministic PIL image from a fixed latent initialisation.

4.3 Captioning

SmolVLM2-500M receives the decoded image(s) with a randomly selected prompt from a pool of 6-7 variants ('Describe what you see...', 'What is happening in this scene?', etc.). Output is trimmed at 80-100 tokens and the 'Assistant:' prefix is stripped.

4.4 Patch Modes

14 patch modes are available, sampled with the weights shown. Each mode defines a different structural relationship between the caption [C], the raw tokens [T], planning/reflection think blocks, and optional instruction framing:

Mode	Structure (weight)
`caption_before`	[C] [T] (5)
`caption_after`	[T] [C] (5)
`agent_then_tokens`	[agent label] [C] [T] (4)
`tokens_then_agent`	[T] [agent label] [C<think>] (4)

`instruct_tokens`	User: ... <think>...</think> [T] (4)
`plan_before`	<think>plan</think> [T] [C] (4)
`reflect_after`	[C] [T] <think>reflect</think> (4)
`replace`	[C] replaces [T] (3)
`instruct_caption_tokens`	User: <think/> [C] [T] (3)
`instruct_tokens_caption`	User: <think/> [T] [C<think>] (3)
`mood_before`	[mood preview line] [T] (3)
`mood_after`	[T] <think>mood reflection</think> (3)
`bookend`	[C] [T] [C<think>] (2)
`mood_before_caption_after`	[mood] [T] <think>mood</think> [C] (2)

4.5 Output Files

File	Contents
captioned/*_captioned.jsonl	Patched records: same schema as flattened, text field updated, has_decoded_visuals flag added
visuals/*_visuals.jsonl	Per-visual sub-dataset: youtube_id, activity_id, token_type, raw_tokens, image_path, caption, patch_mode
visuals/images/<stem>/	Saved representative frames/images as JPEG

5. End-to-End Data Flow

The diagram below shows the complete pipeline from source dataset to training-ready records:

Stage	Input	Output	Script
Download & decompress	HuggingFace .jsonl.gz shards (160 files, ~240 GB)	In-memory line iterator	`process_finevideo.py`
Flatten hierarchy	Nested record (video → scenes → activities)	One record per activity	`process_finevideo.py`
Text augmentation	Raw text fields	Synonym-swapped, stopword-dropped, chunk-permuted text	`process_finevideo.py`
Structured field rendering	props, cast, mood, narrative, context, editing, dynamism	Labelled header blocks with robot/human/cinematic framing	`process_finevideo.py`
Lookahead & planning	Current + next activity context	<think> plan / reflect / mood / instruct blocks	`process_finevideo.py`
Agent framing	Raw agent token string	Past / future / observe perspective label prepended	`process_finevideo.py`
Token dropping (4 passes)	Full video_tokens string	Stochastically subsampled token sequences	`process_finevideo.py`
Dedup & write	Layout blocks	*_flattened.jsonl {text, metadata}	`process_finevideo.py`
Cosmos decode	<cosmos_N> token indices	PIL video frames	`decode_and_caption.py`
Seed2 decode	<seed_N> token indices	PIL image	`decode_and_caption.py`
Caption	PIL frames / image	Natural-language caption (SmolVLM2-500M)	`decode_and_caption.py`
Patch text (14 modes)	Flattened record + caption	Patched text with caption in varied structural position	`decode_and_caption.py`
Write outputs	Patched record + visual info	*_captioned.jsonl *_visuals.jsonl + images/	`decode_and_caption.py`

6. Example Output Records

6.1 Basic record with plan and mood blocks

```
### objects_seen: Camera harness. Canon cameras <think>
Analyzing the current scene. ### scene context: Product review outdoors. I will proceed based on this.
</think> I performed the following motion. <agent_5> <agent_6> </agent>
<cosmos_1> <cosmos_2> </cosmos> Overall, I really like it. <think>
Ok, I have seen Host stands at desk. My plan is to observe Host on forest path.
Let me plan it out by imagining the next scene. </think> ### Title: Review of Pros and Cons
### Keywords: Evaluation of product advantages. Neutral, calm
```

6.2 Instruction framing with caption (after decode_and_caption.py)

```
User: Imagine a scene where Host stands on a forest path taking pictures. <think>
I need to visualize Host on forest path. Generating the sequence. </think>
### visual_observation: A person stands on a forested trail holding a camera harness.
<cosmos_1> <cosmos_2> <cosmos_3> </cosmos> <think>
The feeling of this scene is Neutral, transitions to Confused. </think>
```

6.3 Agent-first framing with reflection

```
I will perform the following motion. ### decoded_scene: A person reviews camera equipment outdoors.
<seed_10> <seed_11> </seed> <agent_8> <agent_9> </agent> <think>
I experienced Host on forest path. The mood I perceived was Neutral, focused. </think>
### Context: A video review of the Cotton Carrier G3 Harness.
```

7. Running the Pipeline

7.1 Dependencies

```
pip install wn tqdm torch diffusers transformers pip install cosmos-tokenizer timm einops safetensors
python -c "import wn; wn.download('oewn:2020')"   # one-time WordNet setup
git clone https://huggingface.co/ontocord/seed2    # seed2 tokenizer
```

7.2 Script 1 — process_finevideo.py

```
python process_finevideo.py # Output: output/*_flattened.jsonl
# Progress: processed_files.txt  (safe to interrupt and resume)
```

7.3 Script 2 — decode_and_caption.py

```
python decode_and_caption.py \     --input_dir output \     --output_dir captioned \
    --vis_dir visuals \     --device cuda # Test a single file first:
python decode_and_caption.py --only_file output/test-00000-of-00008_flattened.jsonl
```

7.4 Hardware Requirements

Component	Requirement
process_finevideo.py	CPU-only. ~4 GB RAM per shard. No GPU needed.
decode_and_caption.py	GPU required (CUDA). Minimum 24 GB VRAM recommended (Cosmos + Seed2 + SmolVLM2 simultaneously)
Disk space	~10 GB working space per shard during download. Output JSONL files are much smaller than source.

7.5 Future Work — Agent Token Decoding

The <agent> token decoder is not yet integrated. When ready, the single function decode_agent_tokens(indices) should be added and called at the placeholder comment # TODO: decode agent tokens to motion description in decode_and_caption.py. The output natural-language motion description replaces [motion sequence] throughout the agent framing templates.

8. Design Rationale

The pipeline is built around three core principles for VLA pretraining:

Perspective diversity

The same underlying scene is rendered from robot, human, and cinematic perspectives, with further variation via WordNet synonyms, label augmentation, and structural shuffle. The model must learn that 'I see camera harness', 'objects_seen: camera harness', and 'props: camera harness' all refer to the same percept.

Temporal grounding

Planning think blocks use real lookahead context from the next activity. Reflection blocks use the current scene's mood. The model learns to associate token sequences with what came before and what comes next — essential for sequential decision-making.

Modality alignment

By decoding cosmos/seed2 tokens to images and captioning them, the pipeline creates explicit alignment signal between discrete visual tokens and natural language. The 14 patch modes vary the structural position of captions relative to tokens so the model learns this alignment from many angles.

Agent embodiment

Agent tokens are the robot's own body. Framing them in past, future, and observation tenses — and occasionally wrapping them in instruction or planning context — embeds the concept that the robot is an actor in the scene, not just a viewer.